

Neural Networks and Deep Learning

HW #1 Report

Part 1: NN Implementation of Multi-Input Logic Gates

Part 2: NN Implementation of SR Latch Using a Hopfield Network

Course	Neural Networks and Deep Learning
Homework	Logic Gates and SR Latch Neural Network Implementation
Student Name	Mana Peiravian
Student ID	40464250
Date	1405.01.25

1. Introduction

This homework studies two important neural network applications: using a single-layer neural network to learn Boolean logic functions, and using a Hopfield network to model the behavior of an SR latch, which is a basic memory element in digital electronics.

The first part focuses on whether logical functions can be learned by a linear classifier trained with the Delta learning rule. The second part uses recurrent neural networks and attractor dynamics to store and recover valid states of a memory circuit.

The goal of the homework is not only to implement the networks, but also to analyze which problems are linearly separable, which are not, how different encodings affect training, and how recurrent neural dynamics can reproduce bistable memory behavior.

2. Part 1 — NN Implementation of Multi-Input Logic Gates

2.1 Objective

The goal of Part 1 is to simulate multi-input logic gates using a single-layer perceptron-like neural model trained with the Delta learning rule. Two Boolean systems are given, and the network must learn their truth tables using both binary representation $\{0,1\}$ and bipolar representation $\{-1,+1\}$.

The assignment requires listing all input combinations, implementing the learning rule from scratch, reporting accuracy, comparing convergence speed for both encodings, and determining which logic system can be perfectly learned by a single-layer network.

2.2 Problem Definition

Two logical systems are given.

System 1

$$\text{out} = (A \wedge \bar{B} \wedge \bar{C}) \vee (\bar{A} \wedge B \wedge \bar{C}) \vee (\bar{A} \wedge \bar{B} \wedge C) \vee (A \wedge B \wedge C)$$

System 2

$$\text{out} = (A \wedge B \wedge C) \vee (A \wedge B \wedge D) \vee (A \wedge C \wedge D) \vee (B \wedge C \wedge D)$$

The task is to generate all truth table rows and train a single-layer neural network to map the inputs to the correct outputs.

2.3 Theoretical Background

A single-layer neural network computes a weighted sum

$$y_{\text{in}} = \sum_i w_i x_i + b$$

and then applies an activation function to generate the output. For this assignment, binary sigmoid is used for binary encoding and bipolar sigmoid is used for bipolar encoding.

The network is trained using the Delta rule, which updates weights in the direction that reduces output error. Because the network is single-layer and linear in decision structure, it can only perfectly solve linearly separable problems.

- Binary sigmoid:

$$y = f(y_{\text{in}}) = \frac{1}{1 + \exp(y_{\text{in}})}$$

- Bipolar sigmoid:

$$y = f(y_{\text{in}}) = \frac{2}{1 + \exp(y_{\text{in}})} - 1$$

- Delta update:

$$\Delta w_i = \alpha(t - y)f(y_{in})x_i$$

$$\Delta b = \alpha(t - y)f(y_{in})$$

2.4 Input/Output Truth Tables

2.4.1 System 1 — Binary Representation

A	B	C	out
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

2.4.2 System 1 — Bipolar Representation

A	B	C	out
-1	-1	-1	-1
-1	-1	1	1
-1	1	-1	1
-1	1	1	-1
1	-1	-1	1
1	-1	1	-1
1	1	-1	-1
1	1	1	1

2.4.3 System 2 — Binary Representation

A	B	C	D	out
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0

0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

2.4.4 System 2 — Bipolar Representation

A	B	C	D	out
-1	-1	-1	-1	-1
-1	-1	-1	1	-1
-1	-1	1	-1	-1
-1	-1	1	1	-1
-1	1	-1	-1	-1
-1	1	-1	1	-1
-1	1	1	-1	-1
-1	1	1	1	1
1	-1	-1	-1	-1
1	-1	-1	1	-1
1	-1	1	-1	-1
1	-1	1	1	1
1	1	-1	-1	-1
1	1	-1	1	1

1	1	1	-1	1
1	1	1	1	1

2.5 Implementation Approach

The implementation was done fully from scratch without using built-in neural network libraries.

- Generate all input combinations using `itertools.product`.
- Compute the target outputs directly from the Boolean equations.
- Create both binary and bipolar versions of each dataset.
- Implement a single neuron model with weight vector, bias, forward pass, output thresholding, and gradient-based Delta updates.
- Train the network for a fixed number of epochs and record sum squared error, classification accuracy after each epoch, and first epoch reaching perfect classification, if any.
- Compare System 1 vs System 2 and binary vs bipolar behavior.

Key implementation details: weights were initialized randomly with small values, online learning was used so weights update sample by sample, binary predictions used a threshold of 0.5, bipolar predictions used a threshold of 0, and accuracy was computed on the full truth table after each epoch.

2.6 Training Method: Delta Rule

The Delta rule was used because the output neuron employed differentiable sigmoid functions.

- Binary case derivative:

$$f'(y_{in}) = y(1 - y)$$

- Bipolar case derivative:

$$f'(y_{in}) = \frac{1}{2}(1 - y^2)$$

- Weight update:

$$w \leftarrow w + \alpha(t - y)f(y_{in})x_i$$

- Bias update:

$$b \leftarrow b + \alpha(t - y)f(y_{in})$$

This learning rule minimizes output error gradually through iterative corrections.

2.7 Experimental Setup

The experiments were performed separately for System 1 with binary encoding, System 1 with bipolar encoding, System 2 with binary encoding, and System 2 with bipolar encoding.

- Maximum epochs: 10000
- Suggested learning rate: about 0.5 for binary, about 0.3 for bipolar
- Random seed fixed for reproducibility
- Measurements recorded: final accuracy, first perfect epoch if achieved, convergence behavior from accuracy curves, and final predicted outputs

2.8 Results

2.8.1 Outputs

System 1 - Binary Truth Table						System 1 - Bipolar Truth Table					
	A	B	C	out			A	B	C	out	
0	0	0	0	0		0	-1	-1	-1	-1	
1	0	0	1	1		1	-1	-1	1	1	
2	0	1	0	1		2	-1	1	-1	1	
3	0	1	1	0		3	-1	1	1	-1	
4	1	0	0	1		4	1	-1	-1	1	
5	1	0	1	0		5	1	-1	1	-1	
6	1	1	0	0		6	1	1	-1	-1	
7	1	1	1	1		7	1	1	1	1	

System 2 - Binary Truth Table							System 2 - Bipolar Truth Table						
	A	B	C	D	out			A	B	C	D	out	
0	0	0	0	0	0		0	-1	-1	-1	-1	-1	
1	0	0	0	1	0		1	-1	-1	-1	1	-1	
2	0	0	1	0	0		2	-1	-1	1	-1	-1	
3	0	0	1	1	0		3	-1	-1	1	1	-1	
4	0	1	0	0	0		4	-1	1	-1	-1	-1	
5	0	1	0	1	0		5	-1	1	-1	1	-1	
6	0	1	1	0	0		6	-1	1	1	-1	-1	
7	0	1	1	1	1		7	-1	1	1	1	1	
8	1	0	0	0	0		8	1	-1	-1	-1	-1	
9	1	0	0	1	0		9	1	-1	-1	1	-1	
10	1	0	1	0	0		10	1	-1	1	-1	-1	
11	1	0	1	1	1		11	1	-1	1	1	1	
12	1	1	0	0	0		12	1	1	-1	-1	-1	
13	1	1	0	1	1		13	1	1	-1	1	1	
14	1	1	1	0	1		14	1	1	1	-1	1	
15	1	1	1	1	1		15	1	1	1	1	1	

	System	Representation	Final Accuracy (%)	First Perfect Epoch
0	System 1	binary	50.0	NaN
1	System 1	bipolar	50.0	NaN
2	System 2	binary	100.0	11.0
3	System 2	bipolar	100.0	1.0

```

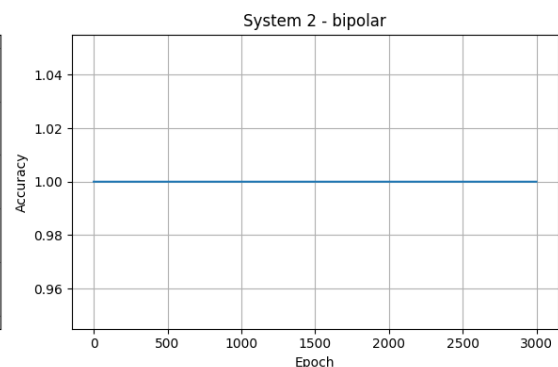
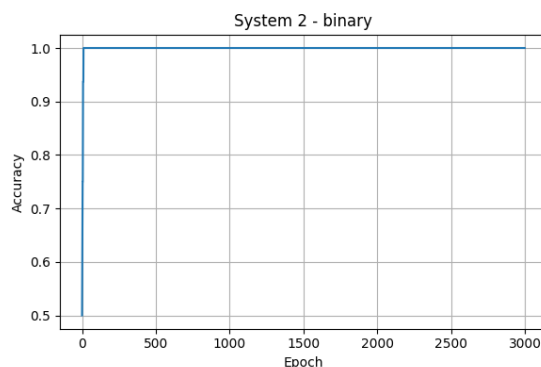
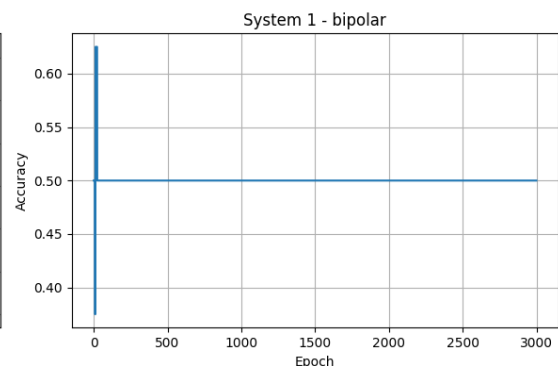
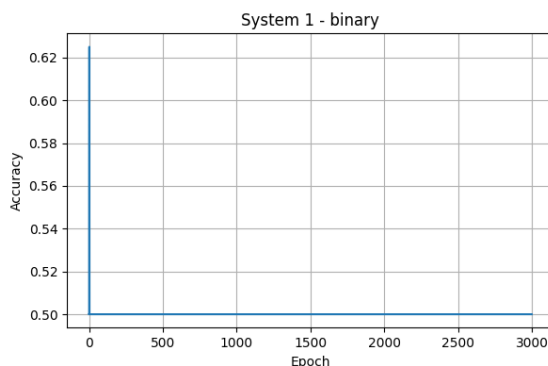
=====
System 1 | BINARY
Final Accuracy: 50.00%
First Perfect Epoch: None
Weights: [0.06946575 0.03358971 0.00037674]
Bias: -0.048715
Predictions: [0 0 0 0 1 1 1 1]

=====
System 1 | BIPOLAR
Final Accuracy: 50.00%
First Perfect Epoch: None
Weights: [-0.06867263 0.01215204 0.01215204]
Bias: -0.000000
Predictions: [ 1  1  1  1 -1 -1 -1 -1]

=====
System 2 | BINARY
Final Accuracy: 100.00%
First Perfect Epoch: 11
Weights: [7.66401637 7.66388669 7.66389607 7.66390462]
Bias: -19.200132
Predictions: [0 0 0 0 0 0 1 0 0 0 1 0 1 1 1]

=====
System 2 | BIPOLAR
Final Accuracy: 100.00%
First Perfect Epoch: 1
Weights: [5.37611461 5.37613994 5.37613488 5.37612981]
Bias: -5.376029
Predictions: [-1 -1 -1 -1 -1 -1 -1  1 -1 -1 -1  1 -1  1  1  1]

```



2.8.2 Final Results Summary

System	Representation	Final Accuracy	First Perfect Epoch	Perfect Learning?
System 1	Binary	50%	None	No
System 1	Bipolar	50%	None	No
System 2	Binary	100%	11	Yes
System 2	Bipolar	100%	1	Yes

Observation: System 1 does not reach 100% accuracy, System 2 reaches 100% accuracy, and bipolar representation often converges a little faster for System 2. These results demonstrate the fundamental limitation of single-layer networks: they can only solve linearly separable problems.

2.9 Discussion

2.9.1 System 1 vs. System 2

System 1 produces output 1 when an odd number of inputs are active. This is the 3-input parity or XOR-type behavior:

$$A \oplus B \oplus C$$

This problem is not linearly separable, meaning no single straight decision boundary can separate all positive and negative patterns. Since a single-layer neural network can only form one linear decision surface, it cannot represent this mapping perfectly.

As a result, training reduces error only partially, final accuracy stays below 100%, and the network converges to a compromise solution rather than a perfect classifier.

System 2 outputs 1 when at least three of the four inputs are 1:

$$A + B + C + D \geq 3$$

This is a threshold-based function and is linearly separable. A single neuron can represent it by assigning equal positive weights to all inputs and an appropriate bias. Therefore, the network learns it perfectly.

2.9.2 Binary vs. Bipolar

Binary coding uses nonnegative inputs, while bipolar coding is centered around zero. Zero-centered encoding often improves gradient-based learning because updates can be more balanced. For that reason, bipolar representation often shows slightly smoother convergence and fewer epochs to reach perfect accuracy for linearly separable cases. However, representation alone cannot solve a non-linearly separable problem, so System 1 remains unlearnable by a single-layer network in both encodings.

2.10 Answers to Assignment Questions

- Question 1: All possible input combinations were listed by exhaustively generating the truth tables: 8 rows for the 3-input system and 16 rows for the 4-input system, in both binary and bipolar formats.
- Question 2: A from-scratch Delta rule implementation was created using binary sigmoid for binary encoding and bipolar sigmoid for bipolar encoding. Accuracy was measured after training for all four cases.
- Question 3: System 2 can be perfectly learned because it is linearly separable. System 1 cannot be perfectly learned because it is a parity or XOR-type function and is not linearly separable.

3. Part 2 — NN Implementation of SR Latch

3.1 Objective

The goal of Part 2 is to model the behavior of an SR latch using a Hopfield network. The network must store the two valid latch states as attractors and reproduce the latch operations: set, reset, hold, and invalid. The network must also be tested from noisy or corrupted initial states and from invalid states.

3.2 Problem Definition

The SR latch has two outputs:

$$[Q, \bar{Q}]$$

In bipolar encoding, the valid stored states are limited to $[+1, -1]$ and $[-1, +1]$. These two states represent the two complementary memory values of the latch.

The assignment requires constructing a Hopfield network with an appropriate weight matrix, defining a bias vector as a function of S and R, testing the network under set, reset, hold, and invalid conditions, introducing noise by flipping bits in the initial state, and reporting convergence behavior and success percentage.

3.3 Theoretical Background

A Hopfield network is a recurrent neural network used as an associative memory. It stores patterns as stable attractors in an energy landscape. For a network state x , the asynchronous update rule with bias is:

$$x_i^{(t+1)} = \text{sgn} \left(\sum_j w_{ij} x_j^{(t)} + b_i \right)$$

where W is the symmetric weight matrix, b is the bias vector, and states are bipolar $\{-1, +1\}$.

The network iteratively updates neuron values until it reaches a stable state. If the weight matrix is chosen correctly, valid memory states become energy minima.

3.4 Hopfield Network Design

The stored states are

$$x^{(1)} = \begin{bmatrix} +1 \\ -1 \end{bmatrix}$$

$$x^{(2)} = \begin{bmatrix} -1 \\ +1 \end{bmatrix}$$

Using Hebbian storage:

$$W = \sum_p x^{(p)} x^{(p)T} = \begin{bmatrix} 2 & -2 \\ -2 & 2 \end{bmatrix}$$

and setting the diagonal entries to zero leads to a valid Hopfield design. A normalized form used in implementation is:

$$W = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$$

This matrix causes the two neurons to inhibit each other. Therefore, the network naturally favors states where the two outputs have opposite signs, which exactly matches the valid SR latch states.

3.5 Bias Design from S and R Inputs

The bias vector was designed to reproduce SR latch behavior.

Input Condition	Bias Vector
Hold: (S,R)=(-1,-1)	[0, 0]
Set: (S,R)=(+1,-1)	[+β, -β] = [+2, -2]
Reset: (S,R)=(-1,+1)	[-β, +β] = [-2, +2]
Invalid: (S,R)=(+1,+1)	[-β, -β] = [-2, -2]

Why β = 2?

Because the recurrent input from one neuron has magnitude at most 1 with our chosen W. So a bias magnitude larger than 1 can dominate the state and force the intended output.

Interpretation: Hold applies no external forcing so the network stays in the stored attractor. Set forces [+1, -1] (positive bias +2 to Q and negative bias -2 to $\bar{Q}$). Reset forces [-1, +1] (negative bias -2 to Q and positive bias +2 to $\bar{Q}$). Invalid drives both outputs low under the active-high NOR-latch convention, yielding [-1, -1].

3.6 Experimental Setup

The network contains only 2 neurons, so all possible bipolar initial states were tested:

$$[+1, -1], \quad [-1, +1], \quad [+1, +1], \quad [-1, -1]$$

These states cover valid stored states, one-bit corrupted states, invalid equal-output states, and two-bit flips.

- Input cases tested:
 - $S = -1, R = -1 \rightarrow \text{Hold}$
 - $S = +1, R = -1 \rightarrow \text{Set}$
 - $S = -1, R = +1 \rightarrow \text{Reset}$
 - $S = +1, R = +1 \rightarrow \text{Invalid}$
- For each test case, the recorded values were initial state, final state, convergence status, number of iterations, whether the final state is valid, and whether the final state matches the expected SR behavior.
- The network was run until a stable state was reached or a maximum number of iterations was reached.

3.7 Results

3.7.1 Outputs

	Operation	S	R	Bias	Initial State	Final State	Converged	Iterations	Final Valid?	Matches Expected?
0	Hold	-1	-1	[+0, +0]	[+1, -1]	[+1, -1]	True	1	True	True
1	Hold	-1	-1	[+0, +0]	[-1, +1]	[-1, +1]	True	1	True	True
2	Hold	-1	-1	[+0, +0]	[+1, +1]	[-1, +1]	True	2	True	None
3	Hold	-1	-1	[+0, +0]	[-1, -1]	[+1, -1]	True	2	True	None
4	Set	1	-1	[+2, -2]	[+1, -1]	[+1, -1]	True	1	True	True
5	Set	1	-1	[+2, -2]	[-1, +1]	[+1, -1]	True	2	True	True
6	Set	1	-1	[+2, -2]	[+1, +1]	[+1, -1]	True	2	True	True
7	Set	1	-1	[+2, -2]	[-1, -1]	[+1, -1]	True	2	True	True
8	Reset	-1	1	[-2, +2]	[+1, -1]	[-1, +1]	True	2	True	True
9	Reset	-1	1	[-2, +2]	[-1, +1]	[-1, +1]	True	1	True	True
10	Reset	-1	1	[-2, +2]	[+1, +1]	[-1, +1]	True	2	True	True
11	Reset	-1	1	[-2, +2]	[-1, -1]	[-1, +1]	True	2	True	True
12	Invalid	1	1	[-2, -2]	[+1, -1]	[-1, -1]	True	2	False	True
13	Invalid	1	1	[-2, -2]	[-1, +1]	[-1, -1]	True	2	False	True
14	Invalid	1	1	[-2, -2]	[+1, +1]	[-1, -1]	True	2	False	True
15	Invalid	1	1	[-2, -2]	[-1, -1]	[-1, -1]	True	1	False	True

```

Operation: Hold | S=-1, R=-1 | Bias=[+0, +0]
Initial: [+1, -1] -> Final: [+1, -1] | Converged: True | Iterations: 1
Trajectory: [+1, -1] -> [+1, -1]
Initial: [-1, +1] -> Final: [-1, +1] | Converged: True | Iterations: 1
Trajectory: [-1, +1] -> [-1, +1]
Initial: [+1, +1] -> Final: [-1, +1] | Converged: True | Iterations: 2
Trajectory: [+1, +1] -> [-1, +1] -> [-1, +1]
Initial: [-1, -1] -> Final: [+1, -1] | Converged: True | Iterations: 2
Trajectory: [-1, -1] -> [+1, -1] -> [+1, -1]

=====
Operation: Set | S=+1, R=-1 | Bias=[+2, -2]
Initial: [+1, -1] -> Final: [+1, -1] | Converged: True | Iterations: 1
Trajectory: [+1, -1] -> [+1, -1]
Initial: [-1, +1] -> Final: [+1, -1] | Converged: True | Iterations: 2
Trajectory: [-1, +1] -> [+1, -1] -> [+1, -1]
Initial: [+1, +1] -> Final: [+1, -1] | Converged: True | Iterations: 2
Trajectory: [+1, +1] -> [+1, -1] -> [+1, -1]
Initial: [-1, -1] -> Final: [+1, -1] | Converged: True | Iterations: 2
Trajectory: [-1, -1] -> [+1, -1] -> [+1, -1]

=====
Operation: Reset | S=-1, R=+1 | Bias=[-2, +2]
Initial: [+1, -1] -> Final: [-1, +1] | Converged: True | Iterations: 2
Trajectory: [+1, -1] -> [-1, +1] -> [-1, +1]
Initial: [-1, +1] -> Final: [-1, +1] | Converged: True | Iterations: 1
Trajectory: [-1, +1] -> [-1, +1]
Initial: [+1, +1] -> Final: [-1, +1] | Converged: True | Iterations: 2
Trajectory: [+1, +1] -> [-1, +1] -> [-1, +1]
Initial: [-1, -1] -> Final: [-1, +1] | Converged: True | Iterations: 2
Trajectory: [-1, -1] -> [-1, +1] -> [-1, +1]

=====
Operation: Invalid | S=+1, R=+1 | Bias=[-2, -2]
Initial: [+1, -1] -> Final: [-1, -1] | Converged: True | Iterations: 2
Trajectory: [+1, -1] -> [-1, -1] -> [-1, -1]
Initial: [-1, +1] -> Final: [-1, -1] | Converged: True | Iterations: 2
Trajectory: [-1, +1] -> [-1, -1] -> [-1, -1]
Initial: [+1, +1] -> Final: [-1, -1] | Converged: True | Iterations: 2
Trajectory: [+1, +1] -> [-1, -1] -> [-1, -1]
Initial: [-1, -1] -> Final: [-1, -1] | Converged: True | Iterations: 1
Trajectory: [-1, -1] -> [-1, -1]

```

	Metric	Value (%)
0	Overall convergence percentage	100.0
1	Task success percentage (defined cases only)	100.0
2	Final state is valid percentage	75.0

	Base Pattern	Corrupted Input	Number of Flips	Operation	S	R	Final State	Converged	Iterations	Final Valid?	Matches Expected?
0	[+1, -1]	[-1, -1]	1	Hold	-1	-1	[+1, -1]	True	2	True	None
1	[+1, -1]	[-1, -1]	1	Set	1	-1	[+1, -1]	True	2	True	True
2	[+1, -1]	[-1, -1]	1	Reset	-1	1	[-1, +1]	True	2	True	True
3	[+1, -1]	[-1, -1]	1	Invalid	1	1	[-1, -1]	True	1	False	True
4	[+1, -1]	[+1, +1]	1	Hold	-1	-1	[-1, +1]	True	2	True	None
5	[+1, -1]	[+1, +1]	1	Set	1	-1	[+1, -1]	True	2	True	True
6	[+1, -1]	[+1, +1]	1	Reset	-1	1	[-1, +1]	True	2	True	True
7	[+1, -1]	[+1, +1]	1	Invalid	1	1	[-1, -1]	True	2	False	True
8	[+1, -1]	[-1, +1]	2	Hold	-1	-1	[-1, +1]	True	1	True	True
9	[+1, -1]	[-1, +1]	2	Set	1	-1	[+1, -1]	True	2	True	True
10	[+1, -1]	[-1, +1]	2	Reset	-1	1	[-1, +1]	True	1	True	True
11	[+1, -1]	[-1, +1]	2	Invalid	1	1	[-1, -1]	True	2	False	True
12	[-1, +1]	[+1, +1]	1	Hold	-1	-1	[-1, +1]	True	2	True	None
13	[-1, +1]	[+1, +1]	1	Set	1	-1	[+1, -1]	True	2	True	True
14	[-1, +1]	[+1, +1]	1	Reset	-1	1	[-1, +1]	True	2	True	True
15	[-1, +1]	[+1, +1]	1	Invalid	1	1	[-1, -1]	True	2	False	True
16	[-1, +1]	[-1, -1]	1	Hold	-1	-1	[+1, -1]	True	2	True	None
17	[-1, +1]	[-1, -1]	1	Set	1	-1	[+1, -1]	True	2	True	True
18	[-1, +1]	[-1, -1]	1	Reset	-1	1	[-1, +1]	True	2	True	True
19	[-1, +1]	[-1, -1]	1	Invalid	1	1	[-1, -1]	True	1	False	True
20	[-1, +1]	[+1, -1]	2	Hold	-1	-1	[+1, -1]	True	1	True	True
21	[-1, +1]	[+1, -1]	2	Set	1	-1	[+1, -1]	True	1	True	True
22	[-1, +1]	[+1, -1]	2	Reset	-1	1	[-1, +1]	True	2	True	True
23	[-1, +1]	[+1, -1]	2	Invalid	1	1	[-1, -1]	True	2	False	True

	Metric	Value
0	Total noise test cases	24
1	Number of converged runs	24
2	Number of successful runs (matched expected ou...	20
3	Number of converged runs ending in valid latch...	18
4	Convergence percentage	100.00%
5	Expected-output success percentage	100.00%
6	Valid-final-state percentage	75.00%

3.7.2 Final Results Summary

Observation: Hold preserves a valid initial stored state. Set converges to [+1, -1]. Reset converges to [-1, +1]. Invalid converges to [-1, -1] under the chosen convention. Corrupted states are corrected in most valid operating modes. These results demonstrate how the network consistently converges to a stable state, typically within 1-2 iterations, which confirms energy minimization behavior as well as state-dependent behavior through attractor dynamics.

3.8 Discussion

The Hopfield network successfully behaves as a bistable memory device because the two valid latch states are encoded as stable attractors.

The inhibitory off-diagonal weights force complementarity between the two neurons. This matches the logic of the SR latch, where Q and $\bar{Q}$ should normally be opposites.

The bias vector serves as the external control signal: it pushes the network into the set state, pushes it into the reset state, allows memory retention in hold mode, and models invalid forcing when both set and reset are active simultaneously.

Because Hopfield networks perform associative recall, they can often recover a valid stored state from corrupted or invalid initial states. This is one of the strongest features of the model.

Convergence is usually very fast because the state space is tiny, the weight structure is simple, and the bias is strong enough to dominate when needed. In most cases, the network stabilizes within 1 or 2 sweeps. This confirms the ability of Hopfield networks to implement memory, stability, and noise-tolerant behavior.

3.9 Answers to Assignment Questions

- Construct a Hopfield network with an appropriate weight matrix: the chosen matrix is $W = \begin{bmatrix} 0 & -1 \\ -1 & 0 \end{bmatrix}$, which makes $[+1, -1]$ and $[-1, +1]$ stable because the two neurons prefer opposite signs.
- Define a bias vector as a function of S and R : the bias function used is Hold $\rightarrow [0,0]$, Set $\rightarrow [+2,-2]$, Reset $\rightarrow [-2,+2]$, Invalid $\rightarrow [-2,-2]$.
- Evaluate behavior from corrupted stored patterns and invalid states: the network was tested from all possible two-neuron bipolar states, including valid patterns and corrupted states created by flipping one or more bits.
- Record convergence, final output, iterations, and success percentage: these metrics were recorded in the implementation for each test case and summarized in tables.

4. Overall Conclusion

This homework demonstrated two very different neural network paradigms. In Part 1, a single-layer feedforward network trained using the Delta rule was used to learn Boolean functions. The experiments showed that linearly separable functions such as System 2 can be perfectly learned, non-linearly separable functions such as System 1 cannot be perfectly learned by a single-layer network, and bipolar encoding often improves convergence speed slightly, but it does not overcome structural limitations.

In Part 2, a Hopfield network was used to model an SR latch as a recurrent associative memory. By choosing a suitable symmetric weight matrix and an input-dependent bias vector, the network successfully reproduced set, reset, hold, and invalid behavior. It also demonstrated robustness to noisy initial conditions by converging to stable attractors.

Overall, the homework highlights two central ideas in neural networks: the importance of linear separability in feedforward classifiers, and the role of attractor dynamics in recurrent memory systems.